

Linear Algebra for ML & AI

Short Lecture Notes

Vectors · Matrices · Linear Equations · Vector Spaces · Eigenvalues · Decompositions · ML Applications

These notes cover the core linear algebra that every ML/AI practitioner needs — from vectors and matrices through SVD and PCA. Each chapter pairs **mathematical intuition** with **NumPy code** and **ML connections**. Prerequisite: basic Python; no prior linear algebra required.

Ch1 Vectors

A **vector** $\mathbf{v} \in \mathbb{R}^n$ is an ordered list of n real numbers. Geometrically it is an arrow in n -dimensional space with direction and magnitude. Vectors are the fundamental data container in ML — every data point, word embedding, and model weight is a vector.

Core Operations

Addition: $(\mathbf{u} + \mathbf{v})_i = u_i + v_i$ **Scaling:** $(c\mathbf{v})_i = cv_i$

Dot Product & Norm

The dot product measures how much two vectors *align*:

$$\text{Dot Product } \mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

The **Euclidean norm** (length) follows directly: $\|\mathbf{v}\| = \sqrt{\mathbf{v} \cdot \mathbf{v}}$. A **unit vector** $\hat{\mathbf{v}} = \mathbf{v} / \|\mathbf{v}\|$ has norm 1.

Two vectors are **orthogonal** iff $\mathbf{u} \cdot \mathbf{v} = 0$ (angle = 90°).

Cosine Similarity

Normalising the dot product gives a scale-free similarity metric:

$$\cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \in [-1, 1]$$

$\cos \theta = 1$: identical direction; 0: orthogonal; -1 : opposite.

Projection

The **projection** of $\mathbf{a}$ onto $\mathbf{b}$ is the shadow of $\mathbf{a}$ along $\mathbf{b}$:

$$\text{proj}_{\mathbf{b}} \mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|^2} \mathbf{b}$$

Span & Linear Independence

The **span** of a set of vectors is all their linear combinations. A set is **linearly independent** if no vector can be written as a combination of the others: $\sum c_i \mathbf{v}_i = \mathbf{0} \Rightarrow c_i = 0 \forall i$.

```
import numpy as np
u, v = np.array([1,2,3]), np.array([4,5,6])
dot = np.dot(u, v)          # 32
norm = np.linalg.norm(v)    # sqrt(77)
cos = dot / (np.linalg.norm(u) * norm)
proj = (np.dot(u,v)/np.dot(v,v)) * v
```

ML Connection Word embeddings (word2vec, GloVe, BERT) represent words as vectors in $\mathbb{R}^{300}$ to $\mathbb{R}^{1024}$. Cosine similarity between vectors captures semantic relatedness: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$.

Ch2 Matrices

A **matrix** $\mathbf{A} \in \mathbb{R}^{m \times n}$ arranges mn numbers in m rows and n columns. Matrices represent *linear transformations*: they rotate, scale, shear, and project vectors.

Transpose

$(\mathbf{A}^\top)_{ij} = A_{ji}$ — rows become columns. Key identity: $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$.

Matrix Multiplication

$$\text{Multiply } (\mathbf{AB})_{ij} = \sum_{k=1}^p A_{ik} B_{kj} \quad \mathbf{A} \in \mathbb{R}^{m \times p}, \mathbf{B} \in \mathbb{R}^{p \times n}$$

Not commutative in general: $\mathbf{AB} \neq \mathbf{BA}$.

Determinant & Trace

Trace: $\text{tr}(\mathbf{A}) = \sum_i A_{ii}$ (sum of diagonal)

Det (2x2): $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$

The determinant measures the *volume scaling factor* of the transformation. If $\det \mathbf{A} = 0$, the transformation collapses space — $\mathbf{A}$ is singular.

Inverse

$\mathbf{AA}^{-1} = \mathbf{I}$ exists iff $\det \mathbf{A} \neq 0$

Properties: $(\mathbf{AB})^{-1} = \mathbf{B}^{-1} \mathbf{A}^{-1}$; $(\mathbf{A}^{-1})^\top = (\mathbf{A}^\top)^{-1}$.

Rank

The **rank** of $\mathbf{A}$ is the dimension of its column space = number of linearly independent columns = number of independent rows.

```
C = A @ B          # matrix multiply
det = np.linalg.det(A)
Ainv = np.linalg.inv(A)
rank = np.linalg.matrix_rank(A)
tr = np.trace(A)
```

ML Connection Every layer of a neural network applies a linear transformation: $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$. The weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is learned by gradient descent.

Ch3 Systems of Linear Equations

A system of m equations in n unknowns is written $\mathbf{Ax} = \mathbf{b}$. Three cases depending on rank:

- **Unique solution:** $\text{rank}(\mathbf{A}) = n$ and $m = n$ (square,

non-singular)

- **No solution:** overdetermined ($m > n$), inconsistent
- **Infinite solutions:** underdetermined ($m < n$ or rank deficient)

Gaussian Elimination

Apply row operations (swap, scale, add multiples) to reach Row Echelon Form, then back-substitute. Complexity: $O(n^3)$ for an $n \times n$ system.

LU Factorization

$A = LU$ L : unit lower-triangular, U : upper-triangular

Once factored, solving $Ax = b$ costs only $O(n^2)$ per right-hand side (forward + back substitution). With pivoting: $PA = LU$.

Least Squares

When $m > n$ and the system is inconsistent, we minimise the residual:

Least Squares Minimise $\|Ax - b\|^2$

Normal equations: $A^T A x = A^T b$

Solution: $x^* = (A^T A)^{-1} A^T b$

The matrix $(A^T A)^{-1} A^T$ is the **Moore-Penrose pseudoinverse**.

```
x = np.linalg.solve(A, b) # square system
x, res, rank, sv = np.linalg.lstsq(A, b, rcond=None)
from scipy.linalg import lu
P, L, U = lu(A) # LU with pivoting
```

ML Connection Linear regression fits $\hat{y} = w^T x$ by solving least squares exactly (normal equations) or by gradient descent. The closed-form solution $w^* = (X^T X)^{-1} X^T y$ is derivable directly from the least squares formulation.

Ch4 Vector Spaces & Orthogonality

A **vector space** is a set closed under addition and scalar multiplication. A **subspace** must contain 0 and be closed under the same operations.

Basis & Dimension

A **basis** is a minimal spanning set of linearly independent vectors. The **dimension** is the number of basis vectors.

Four Fundamental Subspaces

For $A \in \mathbb{R}^{m \times n}$ with rank r :

Subspace	Lives in	Dim
Col space $C(A)$	$\mathbb{R}^m$	r
Null space $N(A)$	$\mathbb{R}^n$	$n - r$
Row space $C(A^T)$	$\mathbb{R}^n$	r
Left null $N(A^T)$	$\mathbb{R}^m$	$m - r$

$C(A) \perp N(A^T)$ and $C(A^T) \perp N(A)$. The **Rank-Nullity theorem**: $r + (n - r) = n$.

Gram-Schmidt Orthogonalisation

Converts any basis into an *orthonormal* basis by iteratively removing projections:

$$\text{Gram-Schmidt } u_i = a_i - \sum_{j < i} \frac{a_i \cdot q_j}{\|q_j\|^2} q_j \quad \text{then } q_i = \frac{u_i}{\|u_i\|}$$

QR Decomposition

Gram-Schmidt on columns of A yields:

$$A = QR \quad Q^T Q = I \text{ (orthonormal cols), } R \text{ upper-triangular}$$

Orthogonal projection onto $C(A)$: $P = QQ^T$, $\hat{b} = Pb$. Error $e = b - \hat{b} \perp C(A)$.

```
Q, R = np.linalg.qr(A)
P = Q @ Q.T # projection matrix
b_hat = P @ b # closest point in C(A)
```

ML Connection QR is the backbone of stable numerical solvers for least squares. Orthogonal bases prevent gradient vanishing in deep networks (orthogonal weight initialisation).

Ch5 Eigenvalues & Eigenvectors

Some vectors only get *stretched or flipped* by A — never rotated. These special directions are the eigenvectors.

Definition $Av = \lambda v$, $v \neq 0$ (λ : eigenvalue, v : eigenvector)

Finding Eigenvalues

Rearrange: $(A - \lambda I)v = 0$ has non-trivial solutions iff:

Characteristic polynomial: $\det(A - \lambda I) = 0$ (n roots in $\mathbb{C}$)

Useful identities: $\text{tr}(A) = \sum \lambda_i$ and $\det(A) = \prod \lambda_i$.

Diagonalisation

If A has n linearly independent eigenvectors (cols of P):

$$A = P \Lambda P^{-1} \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$$

$$\text{Matrix powers: } A^k = P \Lambda^k P^{-1}$$

Spectral Theorem

For **symmetric** matrices ($A = A^T$), eigenvalues are real and eigenvectors are orthogonal:

$$A = Q \Lambda Q^T \quad (Q \text{ orthogonal: } Q^T = Q^{-1})$$

A symmetric matrix is **positive definite** (PD) iff all $\lambda_i > 0$ iff $x^T A x > 0 \forall x \neq 0$.

```
vals, vecs = np.linalg.eig(A)
vals, vecs = np.linalg.eigh(A.sym) # symmetric, stable
# compute A^5 efficiently
A5 = vecs @ np.diag(vals**5) @ vecs.T
```

ML Connection PCA uses the eigenvectors of the covariance matrix as the principal directions. Eigenvalues = variance explained per direction. **PageRank** computes the dominant eigenvector of the web graph transition matrix.

Ch 6 Matrix Decompositions

Matrix decompositions reveal the hidden structure of a matrix and enable efficient, numerically stable computation.

LU Decomposition

$A = LU$ (with pivoting $PA = LU$). Used for solving linear systems, computing determinants and inverses. Cost: $O(n^3)$ once, then $O(n^2)$ per new right-hand side.

QR Decomposition

$A = QR$. Numerically superior to normal equations for least squares. Also the basis of the QR algorithm for finding all eigenvalues.

Cholesky Decomposition

For **symmetric positive definite** matrices:

$$A = LL^T \quad L \text{ real lower-triangular with positive diagonal}$$

Twice as fast as LU. Standard for covariance matrices, Gaussian processes, and Kalman filters.

Singular Value Decomposition (SVD)

The most powerful and general decomposition — works for *any* matrix:

$$SVD A = U \Sigma V^T \quad A \in \mathbb{R}^{m \times n}$$

$U \in \mathbb{R}^{m \times m}$: left singular vectors (orthogonal)

$\Sigma \in \mathbb{R}^{m \times n}$: singular values $\sigma_1 \geq \dots \geq \sigma_r > 0$

$V \in \mathbb{R}^{n \times n}$: right singular vectors (orthogonal)

Best rank- k approximation (Eckart-Young theorem):

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T = U_k \Sigma_k V_k^T \quad \text{minimises } \|A - B\|_F \text{ over all rank-}k \text{ matrices } B$$

Matrix norms via SVD: $\|A\|_2 = \sigma_1$; $\|A\|_F = \sqrt{\sum \sigma_i^2}$; $\kappa(A) = \sigma_{\max} / \sigma_{\min}$.

```
# Thin SVD
U, s, Vt = np.linalg.svd(A, full_matrices=False)
Ak = (U[:, :k] * s[:k]) @ Vt[:, :k] # rank-k approx
# Cholesky
from scipy.linalg import cho_factor, cho_solve
c, low = cho_factor(A.pd)
x = cho_solve((c, low), b)
```

ML Connection SVD is central to: **Recommender systems** (Netflix-style matrix factorisation), **NLP** (Latent Semantic Analysis, word embeddings via co-occurrence SVD), **Image compression** (keep top- k singular values), **PCA** (principal components = right singular vectors).

Ch 7 Applications in ML & AI

Principal Component Analysis (PCA)

PCA finds the directions of maximum variance in data.

1. Centre: $\tilde{X} = X - \bar{x}$
2. Covariance: $C = \frac{1}{n} \tilde{X}^T \tilde{X} \in \mathbb{R}^{d \times d}$
3. Eigen-decomp: $C = Q \Lambda Q^T$
4. Project: $Z = \tilde{X} Q_k \in \mathbb{R}^{n \times k}$

Equivalently via thin SVD: $\tilde{X} = U \Sigma V^T$, principal components = cols of V , scores = $U \Sigma$.

Linear Regression

Fit $\hat{y} = Xw$ by minimising the squared loss:

$$\mathcal{L}(w) = \|Xw - y\|^2 \Rightarrow w^* = (X^T X)^{-1} X^T y$$

In practice: use `np.linalg.lstsq` (QR-based, numerically stable) instead of computing $(X^T X)^{-1}$ directly.

Recommender Systems

Model ratings matrix $R \in \mathbb{R}^{m \times n}$ (users $\times$ items) as a low-rank product:

$$R \approx UV^T \quad U \in \mathbb{R}^{m \times k}, V \in \mathbb{R}^{n \times k}$$

Minimise $\sum_{(i,j) \in \Omega} (R_{ij} - u_i \cdot v_j)^2$ via ALS or SGD

Neural Networks

A deep network is a composed sequence of linear + non-linear operations: $h^{(l)} = \sigma(W^{(l)} h^{(l-1)} + b^{(l)})$.

Backpropagation = chain rule applied to matrix products. Gradient $\partial \mathcal{L} / \partial W^{(l)}$ involves matrix-vector products at every layer.

Attention Mechanism (Transformers)

The self-attention operation is entirely linear algebra:

$$\text{Scaled Dot-Product Attention} \text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

QK^T computes dot-product similarity between all query-key pairs. Dividing by $\sqrt{d_k}$ prevents softmax saturation.

```
from sklearn.decomposition import PCA
Z = PCA(n_components=2).fit_transform(X) # PCA
w = np.linalg.lstsq(X, y, rcond=None)[0] # linear reg
U, s, Vt = np.linalg.svd(R, full_matrices=False)
R_hat = (U[:, :k] * s[:k]) @ Vt[:, :k] # reco
```

ML Connection Almost every algorithm in modern ML/AI is fundamentally linear algebra: gradient descent updates weights via vector addition, batch normalisation uses mean/covariance, attention is matrix multiplication, and PCA / SVD appear in pre-training, compression, and interpretability. **Linear algebra is the language of AI.**

Linear Algebra for ML & AI · Notebooks: 01-vector.ipynb → 07-applications-in-ml-and-ai.ipynb · uv sync && jupyter lab